這是一份為 **YTP (Young Turing Program) 決賽** 量身打造的 **「紙本攜帶型演算法競賽筆記與代碼庫」** 。內容涵蓋 YTP 8 年考古題中最常出現與必備的高頻演算法 。

本報告分為 **7 大核心板塊**，每個主題皆包含 **詳細實作觀念、空間/時間複雜度** 以及 **可以直接抄寫/使用的 C++ 完整範例程式碼**。

目錄

1. **基礎模擬與幾何數學 (Simulation & Geometry)**
2. **搜尋與分治演算法 (Search & Divide-and-Conquer)**
3. **基礎與進階資料結構 (Data Structures)**
4. **圖論演算法 (Graph Theory)**
5. **動態規劃 (Dynamic Programming, DP)**
6. **數論與組合數學 (Number Theory & Combinatorics)**
7. **字串處理演算法 (String Algorithms)**

---

1. 基礎模擬與幾何數學 (Simulation & Geometry)

1.1 座標旋轉與三角函數應用

- 💡 **這在解決什麼問題？**
  在地圖地塊、圖形繪製或機器人旋轉時，給定一個點 (x, y)，想知道把它繞著原點逆時針旋轉 θ 度後的新座標 (x', y')。
- 🧠 **直覺與核心思路**：
- **極座標視角**：任何點都可以看成是「離原點距離為 r，與 X 軸夾角為 α」。所以原座標是 $x = r\cos\alpha$, $y = r\sin\alpha$。
- **旋轉**：轉了 θ 度之後，新夾角變成 $(\alpha + \theta)$。
- **和角公式拆解**：根據高中數學 $\cos(\alpha+\theta) = \cos\alpha\cos\theta - \sin\alpha\sin\theta$，把 r 乘進去替換掉，就能導出：

$$\begin{cases} x' = x\cos\theta - y\sin\theta \\ y' = x\sin\theta + y\cos\theta \end{cases}$$

在座標地圖或繪圖模擬題（如 Flag 旗幟繪製 ）中，經常需要將點 (x, y) 繞原點順時針/逆時針旋轉角度 θ。

- **公式觀念**：
- **逆時針旋轉 θ**：

$$x' = x \cdot \cos\theta - y \cdot \sin\theta$$

$$y' = x \cdot \sin\theta + y \cdot \cos\theta$$

- **極座標轉直角座標**：

$$x = r \cdot \cos\theta, \quad y = r \cdot \sin\theta$$

- C++ 中 `cmath` 的 `acos(-1.0)` 代表 π，`sin`/`cos` 接受弧度（$rad = deg \times \frac{\pi}{180}$）。

```cpp
#include <iostream>
#include <cmath>
#include <iomanip>

using namespace std;

const double PI = acos(-1.0);

// 將點 (x, y) 逆時針旋轉 deg 度
pair<double, double> rotatePoint(double x, double y, double deg) {
    double rad = deg * PI / 180.0;
    double nx = x * cos(rad) - y * sin(rad);
    double ny = x * sin(rad) + y * cos(rad);
    return {nx, ny};
}

int main() {
    double x = 10.0, y = 0.0, angle = 45.0;
    auto [nx, ny] = rotatePoint(x, y, angle);
    cout << fixed << setprecision(2) << nx << " " << ny << "\n";
    return 0;
}
```

1.2 二維向量外積 (Cross Product) 與凸包 (Convex Hull - Andrew's Monotone Chain)

- 💡 **這在解決什麼問題？**
  想像桌面上釘了許多釘子（點），拿一張**橡皮筋**放開，橡皮筋包住最外圍所有釘子所形成的**最小凸多邊形**，就是凸包。
- 🧠 **直覺與核心思路**：

1. **向量外積（判斷轉彎方向）**：

- 我們從點 O 走到 A，再從 A 走到 B。如何知道是在「左轉」還是「右轉」？
- **外積公式**：$\vec{OA} \times \vec{OB} = x_A y_B - y_A x_B$。
- **外積** > 0：往左轉（逆時針）。
- **外積** < 0：往右轉（順時針）。
- **外積** $= 0$：三點在一條直線上。

2. **凸包的維護（Andrew's Monotone Chain）**：

- 凸包的輪廓有個特點：**順著邊界走，只能一直往同一個方向轉彎（不能凹進去）**。
- **步驟**：

1. 先把所有點按照 x 座標從小到大排序（x 一樣就比 y）。
2. 從最左邊開始，依序將點加入輪廓。
3. 如果新加進來的點，會讓最後三點形成「右轉/凹陷」，說明前面的點**被包在裡面了，不是凸包頂點**！立刻把前面的點剔除，直到重新保持「順暢的左轉」。

4. 做了兩次（一次做下半部，一次做上半部），拼起來就是完美凸包。

- **觀念**：
- **外積**：$\vec{A} \times \vec{B} = A_x B_y - A_y B_x$。
- 外積 > 0：$\vec{B}$ 在 $\vec{A}$ 的左側（逆時針折角）。
- 外積 < 0：$\vec{B}$ 在 $\vec{A}$ 的右側（順時針折角）。
- 外積 $= 0$：三點共線。
- **時間複雜度**：$O(N \log N)$（主要為排序時間）。

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Point {
    long long x, y;
    bool operator<(const Point& p) const {
        if (x != p.x) return x < p.x;
        return y < p.y;
    }
};

// 計算外積 (O->A) x (O->B)
long long cross_product(Point O, Point A, Point B) {
    return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
}

// Andrew's Monotone Chain 求凸包
vector<Point> convex_hull(vector<Point>& pts) {
    int n = pts.size(), k = 0;
    if (n <= 3) return pts;
    vector<Point> h(2 * n);
    sort(pts.begin(), pts.end());

    // 下凸包
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && cross_product(h[k-2], h[k-1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    // 上凸包
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross_product(h[k-2], h[k-1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
```

```
}
```

2. 搜尋與分治演算法 (Search & Divide-and-Conquer)

2.1 二分搜答案 (Binary Search on Answer)

- 💡 **這在解決什麼問題？**
  當你要求「最大值的最小值」或「最小值的最大值」，且直接計算答案非常困難時。
- 🧠 **直覺與核心思路**：
- **猜數字遊戲**：想像終極密碼。直接算答案很難，但如果問你：「答案是 100 時，可行還是不可行？」卻非常容易檢查。
- **單調性（關鍵條件）**：必須滿足「如果長度 K 可以，那所有比 K 小的一定都可以」。
- **邏輯步驟**：

1. 設定答案的範圍區間 $[low, high]$。
2. 取中間值 $mid = (low + high)/2$。
3. 寫一個判定函數 `check(mid)` 測試這個 mid 能不能滿足題目需求。
4. 如果**可以**：代表答案可能可以更大（或更小），將底限 low 調高，繼續折半猜！
5. 如果**不行**：代表 mid 太誇張了，將上限 $high$ 調低。

- 複雜度從窮舉的 $O(N)$ 降到驚人的 $O(\log N)$。

當問題具有**單調性**（例如：「當長度為 K 時可行，則所有 $< K$ 的長度也一定可行」），即可將求極值的問題轉換為判定問題。

- **範例**：尋找滿足 `check(mid) == true` 的最大/最小值。
- **時間複雜度**：$O(f(N) \cdot \log(\text{Range}))$。

```cpp
#include <iostream>
#include <vector>

using namespace std;

bool check(long long val, const vector<long long>& nums, int k) {
    // 判定邏輯：給定參數 val 是否能滿足條件
    long long count = 0;
    for (auto x : nums) count += x / val;
    return count >= k;
}

long long binarySearchAnswer(long long low, long long high, const vector<long
long>& nums, int k) {
    long long ans = low;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (check(mid, nums, k)) {
```

```
            ans = mid;      // 儲存目前可能解答
            low = mid + 1; // 嘗試找更大的值
        } else {
            high = mid - 1;
        }
    }
    return ans;
}
```

2.2 二維網格搜尋 (BFS & DFS)

- 💡 **這在解決什麼問題？**
  在網格迷宮中找出口、計算島嶼面積或尋找最短步數。
- 🧠 **直覺與核心思路**：
- **BFS (廣度優先搜尋) —— 水波擴散法**：
- **邏輯**：像是一滴墨水滴在起點，第一秒擴散到距離 1 的所有格子，第二秒擴散到距離 2 的所有格子.....
- **工具**：使用 **Queue (排隊隊列，先到先處理)**。
- **特點**：只要邊的權重都一樣（每走一步耗時 1），**BFS 第一次碰到終點時，絕對就是「最短路徑」**！
- **DFS (深度優先搜尋) —— 撞牆才回頭法**：
- **邏輯**：像走迷宮時手貼著右邊牆壁一直往前走，走到死胡同才退回上一個叉路口換條路試。
- **工具**：使用 **遞迴 (Recursion / Stack)**。
- **特點**：適合用來找「共有幾種走法」或「標記整片連通的區域（如計算島嶼數量）」。

網格方向數組常採 `dx[] = {-1, 1, 0, 0}`, `dy[] = {0, 0, -1, 1}` 進行上、下、左、右走訪。

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

int bfs_grid(int startX, int startY, int targetX, int targetY, int R, int C, const
vector<vector<int>>& grid) {
    vector<vector<int>> dist(R, vector<int>(C, -1));
    queue<pair<int, int>> q;

    q.push({startX, startY});
    dist[startX][startY] = 0;
```

```
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();

        if (x == targetX && y == targetY) return dist[x][y];

        for (int i = 0; i < 4; ++i) {
            int nx = x + dx[i];
            int ny = y + dy[i];

            if (nx >= 0 && nx < R && ny >= 0 && ny < C && grid[nx][ny] == 0 &&
dist[nx][ny] == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
    return -1; // 無法抵達
}
```

3. 基礎與進階資料結構 (Data Structures)

3.1 並查集 (Disjoint Set Union, DSU)

- 💡 **這在解決什麼問題？**
  有許多小團體，需要隨時把兩個團體合并（Merge），且能以極快速度回答「小明和小華是不是在同一個朋友圈？」（Find）。
- 🧠 **直覺與核心思路**：
- **幫派認老大**：
- 每個朋友圈/幫派都有一個「最高幫主」。
- 如果兩個人去查自己的最高幫主是同一個人，代表他們在同一個幫派。
- **幫派合併**：幫派 A 與幫派 B 合併時，只要讓 A 的幫主認 B 的幫主當老大就行了！
- **神級優化——路徑壓縮 (Path Compression)**：
- 如果幫派階級太長（A認B，B認C，C認D...），查詢會變慢。
- 所以每次幫眾去詢問「我的最高幫主是誰？」的過程中，**順便把路上所有手下的老大直接改成最高幫主**（扁平化管理）。下次再問，一步就能得到答案！
- **觀念**：用於維護元素的動態連通性，含**路徑壓縮 (Path Compression)** 與 **按秩合併 (Union by Rank)**。
- **時間複雜度**：單次操作接近 $O(\alpha(N)) \approx O(1)$。

```cpp
#include <vector>
#include <numeric>

using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) {
        parent.resize(n + 1);
        iota(parent.begin(), parent.end(), 0);
        sz.assign(n + 1, 1);
    }

    int find(int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent[i]); // 路徑壓縮
    }

    bool unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);
        if (root_i != root_j) {
            if (sz[root_i] < sz[root_j]) swap(root_i, root_j);
            parent[root_j] = root_i;
            sz[root_i] += sz[root_j];
            return true;
        }
        return false;
    }

    bool same(int i, int j) {
        return find(i) == find(j);
    }
};
```

3.2 樹狀陣列 (Binary Indexed Tree / Fenwick Tree)

- 💡 **這在解決什麼問題？**
  有一個陣列，裡面的數字會隨時被修改，又需要頻繁查詢「第 1 到第 K 個數字的總和（前綴和）」。
- 🧠 **直覺與核心思路**：
- **為什麼普通做法不行？**
- 普通陣列：修改 $O(1)$，但求和要一個個加，需要 $O(N)$。
- 前綴和陣列：求和 $O(1)$，但每次修改一個數字，後面所有的前綴和都要改，需要 $O(N)$。
- **BIT 的巧妙解法（二進位分工合作）**：

- 任何整數都可以拆成 2 的次方和（如 $13 = 8 + 4 + 1$）。
- BIT 讓每個位置 i 不只存自己，而是負責管轄「長度為 $lowbit(i)$」的一段區間和。（$lowbit(i)$ 就是 i 的二進位中最右邊的 1 所代表的值）。
- **查詢**：要算 $1 \sim 13$ 的和，只要加 `tree[13] + tree[12] + tree[8]`，跳三次就加完了！
- **修改**：修改某個數字時，也只需要順著進位鏈向上更新少數幾個節點。時間複雜度皆為 $O(\log N)$。
- **觀念**：高效維護前綴和、單點修改與區間查詢。
- **時間複雜度**：單次修改/查詢均為 $O(\log N)$。

```
#include <vector>

using namespace std;

struct FenwickTree {
    int n;
    vector<long long> tree;

    FenwickTree(int n) : n(n), tree(n + 1, 0) {}

    // 單點增加 val
    void add(int i, long long val) {
        for (; i <= n; i += i & -i)
            tree[i] += val;
    }

    // 查詢前綴和 [1...i]
    long long query(int i) {
        long long sum = 0;
        for (; i > 0; i -= i & -i)
            sum += tree[i];
        return sum;
    }

    // 區間查詢 [l...r]
    long long queryRange(int l, int r) {
        return query(r) - query(l - 1);
    }
};
```

3.3 線段樹 (Segment Tree) - 單點修改與區間和

- 💡 **這在解決什麼問題？**
  比 BIT 更強大！它可以處理**區間修改**（把區間 $[L, R]$ 都加 5），又能查詢**區間任意資訊**（例如區間最大值、最小值、GCD、區間和）。
- 🧠 **直覺與核心思路**：

- **樹狀分治觀念**：
- 將整個陣列 $[1, N]$ 當作根節點。
- 平分成左右兩半：左子樹管 $[1, N/2]$，右子樹管 $[N/2+1, N]$，一直切切切，直到葉子節點管單個數字。
- **區間查詢**：
- 如果你要查 $[2, 5]$，線段樹會把這個區間拆成樹上**少數幾個已經算好的大區間塊**拼起來。不需要一個個點算，最多只要看 $O(\log N)$ 個樹節點。
- **觀念**：解決動態區間極值 (RMQ)、區間和等靈活維護問題。
- **時間複雜度**：建樹 $O(N)$，修改/查詢 $O(\log N)$。

```
#include <vector>

using namespace std;

class SegmentTree {
    int n;
    vector<long long> tree;

    void build(const vector<long long>& arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }
        int mid = (start + end) / 2;
        build(arr, 2 * node, start, mid);
        build(arr, 2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    void update(int node, int start, int end, int idx, long long val) {
        if (start == end) {
            tree[node] = val;
            return;
        }
        int mid = (start + end) / 2;
        if (idx <= mid) update(2 * node, start, mid, idx, val);
        else update(2 * node + 1, mid + 1, end, idx, val);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }

    long long query(int node, int start, int end, int l, int r) {
        if (r < start || end < l) return 0;
        if (l <= start && end <= r) return tree[node];
        int mid = (start + end) / 2;
        return query(2 * node, start, mid, l, r) + query(2 * node + 1, mid + 1,
end, l, r);
    }
```

```
public:
    SegmentTree(const vector<long long>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(arr, 1, 0, n - 1);
    }

    void update(int idx, long long val) { update(1, 0, n - 1, idx, val); }
    long long query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
```

---

4. 圖論演算法 (Graph Theory)

4.1 Dijkstra 最短路徑演算法（帶權非負圖）

- 💡 **這在解決什麼問題？**
  地圖上許多地點由道路連接，每條路有不同的花費時間（必須為非負數），求從起點到所有點的最短時間。
- 🧠 **直覺與核心思路**：
- **貪婪策略 + 鬆弛 (Relaxation)**：

1. 記住：**「目前離起點最近的那個未確定點，它的最短距離已經不可能更短了！」**
2. 初始化：起點距離為 0，其他點都是無窮大。
3. 用 **Priority Queue (優先隊列/小頂堆)** 隨時取出當前離起點最近的點 u。
4. 拿點 u 當作中繼站，去檢查 u 的所有鄰居 v：如果 從起點到u的距離 + u到v的距離 < 原本記錄到v的距離，就更新 v 的距離（這稱為鬆弛）。
5. 重複這一步驟，直到所有點都處理完畢。

- **觀念**：利用 `priority_queue` 維護當前離源點最近的節點。
- **時間複雜度**：$O((V+E)\log V)$。

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const long long INF = 1e18;

struct Edge {
    int to;
    long long weight;
};

vector<long long> dijkstra(int start, int n, const vector<vector<Edge>>& adj) {
    vector<long long> dist(n + 1, INF);
```

```
    // {距離, 點}
    priority_queue<pair<long long, int>, vector<pair<long long, int>>,
greater<pair<long long, int>>> pq;

    dist[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) continue;

        for (const auto& edge : adj[u]) {
            if (dist[u] + edge.weight < dist[edge.to]) {
                dist[edge.to] = dist[u] + edge.weight;
                pq.push({dist[edge.to], edge.to});
            }
        }
    }
    return dist;
}
```

4.2 拓撲排序 (Topological Sort / Kahn's Algorithm)

- 💡 **這在解決什麼問題？**
  許多任務有「先修條件」（例如：必須先穿襪子才能穿鞋子、必須先學微積分才能學工程數學），求出一種合法的執行順序。
- 🧠 **直覺與核心思路**：
- **入度 (In-degree) 觀念**：
- 「入度」代表有幾門先修課還沒修。
- 如果某門課的**入度** $= 0$，代表它**沒有任何先決條件**，現在就能立刻修！
- **步驟**：

1. 算出所有課程的入度。
2. 把所有入度 $= 0$ 的課通通丟進 Queue 隊列裡。
3. 從 Queue 拿出一門課修掉，並將這門課指向的所有後續課程的「入度減 1」。
4. 如果某門後續課程的入度被減到 0 了，代表它的先條件都滿足了，丟進 Queue 裡！
5. 重複直到課修完。若修完的總數小於總課數，說明課程之間有**死鎖（循環依賴）**。

- **觀念**：計算入度 (Indegree)，將入度為 0 的節點入隊列。
- **時間複雜度**：$O(V + E)$。

```
#include <vector>
#include <queue>
```

```cpp
using namespace std;

vector<int> topological_sort(int n, const vector<vector<int>>& adj, vector<int>&
in_degree) {
    queue<int> q;
    for (int i = 1; i <= n; ++i) {
        if (in_degree[i] == 0) q.push(i);
    }

    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);

        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) q.push(v);
        }
    }

    if ((int)order.size() != n) return {}; // 存在環
    return order;
}
```

4.3 樹上最近公共祖先 (LCA - 倍增法)

- 💡 **這在解決什麼問題？**
  在一棵家族樹中，給定兩個人 A 和 B，找出他們在樹上「最近的共同祖先」是誰。
- 🧠 **直覺與核心思路：**
- **傳統笨方法**：讓較深的人一步一步往上爬，直到兩人碰頭（太慢）。
- **倍增法（二進位跳躍）**：
- 預先讓每個人記住自己往上跳 $1, 2, 4, 8, 16, \ldots, 2^k$ 步的祖先是誰（這可以用 DP 預處理）。
- **查詢兩步驟：**

1. **對齊高度**：先把比較深的那個人，利用二進位跳躍（例如一次跳 8 步，再跳 2 步）直接拉到跟另一個人相同的深度。
2. **一起向上跳**：兩個人同時嘗試向上跳最大的步數（例如 2^k 步）。**只要跳過去的地方不同，就代表還沒過頭**，放膽跳過去！最後他們頭頂上的那一格，就是最近公共祖先。

- **觀念**：`up[u][i]` 表示從點 u 向上走 2^i 步所到達的祖先。
- **預處理與查詢複雜度**：預處理 $O(N \log N)$，單次查詢 $O(\log N)$。

```cpp
#include <vector>

using namespace std;

const int LOGN = 20;
```

```
vector<vector<int>> adj;
vector<vector<int>> up;
vector<int> depth;

void dfs(int u, int p, int d) {
    depth[u] = d;
    up[u][0] = p;
    for (int i = 1; i < LOGN; ++i) {
        up[u][i] = up[up[u][i - 1]][i - 1];
    }
    for (int v : adj[u]) {
        if (v != p) dfs(v, u, d + 1);
    }
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);

    // 將 u 拉到與 v 同高度
    for (int i = LOGN - 1; i >= 0; --i) {
        if (depth[u] - (1 << i) >= depth[v]) {
            u = up[u][i];
        }
    }

    if (u == v) return u;

    // 一起向上倍增跳躍
    for (int i = LOGN - 1; i >= 0; --i) {
        if (up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    }

    return up[u][0];
}
```

5. 動態規劃 (Dynamic Programming, DP)

5.1 0/1 背包問題

- 💡 **這在解決什麼問題？**
  背包限重 W，有 N 件物品，每件物品有各自的重量與價值，且每件物品只能選擇「拿 (1)」或「不拿 (0)」，求能裝下的最大總價值。
- 🧠 **直覺與核心思路**：
- **大問題化小**：考慮第 i 個物品時，對於目前的背包容量 w，我們只有兩種選擇：

1. **不拿第 i 個物品**：最大價值就是「只考慮前 $i-1$ 個物品、容量為 w 時的價值」。

2. **拿第 i 個物品**（前提是背包含得下）：最大價值就是「第 i 個物品的價值 + 只考慮前 $i-1$ 個物品且扣除該物品重量後（容量 $w - weight[i]$）的最大價值」。

- **轉移公式**：`dp[w] = max(不拿的價值, 拿的價值)`。
- **空間優化技巧**：陣列可以簡化成一維，但容量 w 必須**倒著（從 W 扣到重量）更新**，這樣才不會讓同一個物品被重複拿取！
- **觀念**：給定 N 個物品，各具重量與價值，限制承重 W。滾動陣列逆序轉移。
- **時間複雜度**：$O(N \cdot W)$，**空間複雜度**：$O(W)$。

```
#include <vector>
#include <algorithm>

using namespace std;

int knapsack_01(int W, const vector<int>& weights, const vector<int>& values) {
    vector<int> dp(W + 1, 0);
    int n = weights.size();

    for (int i = 0; i < n; ++i) {
        for (int w = W; w >= weights[i]; --w) { // 逆序更新
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[W];
}
```

5.2 最長遞增子序列 ($O(N \log N)$ LIS)

- 💡 **這在解決什麼問題？**
  在一串數字中（例如 `[10, 9, 2, 5, 3, 7, 101]`），挑選出順序不變且數字嚴格遞增的最長子序列（答案如 `[2, 3, 7, 101]`，長度為 4）。
- 🧠 **直覺與核心思路**：
- **貪婪思考**：如果我們想讓遞增子序列盡可能變長，**序列結尾的數字越小越好**（因為結尾越小，後面越容易接新數字）！
- **維護一個 `tails` 陣列**：`tails[i]` 記錄「長度為 $i+1$ 的遞增子序列中，最小的結尾數字」。
- **遍歷每個數字 x**：
- 如果 x 比 `tails` 裡面所有數字都大：太棒了！直接把 x 接在最後面，總長度 $+1$。
- 如果 x 沒有比較大：用**二分搜尋**在 `tails` 中找到第一個 $\geq x$ 的位置，用 x 替換掉它。（這不會縮短當前的長度，但能讓該長度的結尾變小，為後面的數字創造機會！）
- **觀念**：維護一個動態陣列 `tails`，其中 `tails[i]` 存放長度為 $i+1$ 的 LIS 最小結尾值。

```cpp
#include <vector>
#include <algorithm>

using namespace std;

int lengthOfLIS(const vector<int>& nums) {
    vector<int> tails;
    for (int x : nums) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) {
            tails.push_back(x);
        } else {
            *it = x;
        }
    }
    return tails.size();
}
```

5.3 狀態壓縮 DP (Bitmask DP) - 例：TSP 問題

- 💡 **這在解決什麼問題？**
  例如旅行推銷員問題 (TSP)：有 N 個城市（N 很小，如 $N \leq 18$），要走遍所有城市，求最短總距離。
- 🧠 **直覺與核心思路**：
- **用二進位當作「開關手環」**：
- 傳統上記錄「哪些城市去過」需要用陣列，但在位元運算中，一個**整數**的二進位就能代表狀態！
- 例如：整數 13 的二進位是 $(1101)_2$，代表第 0、2、3 個城市已經去過了，第 1 個城市還沒去過。
- **DP 狀態**：`dp[mask][i]` 表示「目前走過的城市集合為 `mask`，且最後停留在城市 `i` 時的最短距離」。
- 利用位元運算 `mask | (1 << v)` 可以快速將城市 v 標記為已去過，將指數級的搜尋優化為多項式時間。
- **觀念**：以整數位元 (Bit Set) 表示當前集合選擇狀態，`dp[mask][i]` 表示走過集合 `mask` 且目前在點 `i` 的極值。
- **時間複雜度**：$O(2^N \cdot N^2)$，適合 $N \leq 18 \sim 20$ 的問題。

```cpp
#include <vector>
#include <algorithm>

using namespace std;

const int INF = 1e9;

int tsp(int n, const vector<vector<int>>& dist) {
```

```
    // dp[mask][i]
    vector<vector<int>> dp(1 << n, vector<int>(n, INF));
    dp[1][0] = 0; // 起始點從 0 開始

    for (int mask = 1; mask < (1 << n); ++mask) {
        for (int u = 0; u < n; ++u) {
            if (!(mask & (1 << u))) continue;
            if (dp[mask][u] == INF) continue;

            for (int v = 0; v < n; ++v) {
                if (!(mask & (1 << v))) {
                    int next_mask = mask | (1 << v);
                    dp[next_mask][v] = min(dp[next_mask][v], dp[mask][u] + dist[u]
[v]);
                }
            }
        }
    }

    int ans = INF;
    for (int i = 1; i < n; ++i) {
        ans = min(ans, dp[(1 << n) - 1][i] + dist[i][0]);
    }
    return ans;
}
```

---

6. 數論與組合數學 (Number Theory & Combinatorics)

6.1 快速冪與模逆元 (Fast Exponentiation & Modular Inverse)

- 💡 **這在解決什麼問題？**

1. 計算 $a^b \pmod M$，當 b 高達 10^9 時，不能連乘 b 次。
2. 計算 $(A/B) \pmod M$ 時，取模運算**不能直接做除法**！

- 🧠 **直覺與核心思路**：
- **快速冪（分治平方）**：
- 求 a^{13}：把 13 拆成二進位 $(1101)_2 = 8 + 4 + 1$。
- 所以 $a^{13} = a^8 \times a^4 \times a^1$。
- 每次把底數自己平方（$a \to a^2 \to a^4 \to a^8$），只需 $\log b$ 次乘法就能算完！
- **模逆元（把除法變乘法）**：
- 除以 B 在模運算中，等於乘以 B 的「模逆元 B^{-1}」。
- 根據**費馬小定理**（當模數 P 是質數時）：$B^{P-1} \equiv 1 \pmod P$。

- 兩邊同除以 B 得：$B^{P-2} \equiv B^{-1} \pmod P$。
- **結論**：想算除以 B，只要用快速冪算出 $B^{P-2} \pmod P$ 乘進去就行了！
- **觀念**：
- **快速冪**：以 binary 表示指數，達到 $O(\log N)$ 取模乘冪。
- **費馬小定理求逆元**：若 P 為質數，則 $a^{-1} \equiv a^{P-2} \pmod P$。

```
#include <iostream>

using namespace std;

long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

// 模逆元
long long modInverse(long long n, long long mod) {
    return power(n, mod - 2, mod);
}
```

6.2 組合數 $C(n,k) \pmod P$ 預處理

- 💡 **這在解決什麼問題？**
  競賽中需要頻繁回答 $C(n,k) = \frac{n!}{k!(n-k)!} \pmod P$ 的數值。
- 🧠 **直覺與核心思路**：
- **空間換時間**：

1. 開場先把 $1! \sim N!$ 的餘數通通算好存進 `fact[]` 陣列（花費 $O(N)$）。
2. 用快速冪算出 $N!$ 的模逆元，再逆向推導算出所有階乘的模逆元 `invFact[]`。

- $O(1)$ **瞬間回答**：
  當詢問 $C(n,k)$ 時，直接查表計算：

$$\text{ans} = \text{fact}[n] \times \text{invFact}[k] \times \text{invFact}[n-k] \pmod P$$

- **觀念**：預處理階乘與階乘逆元，可在 $O(1)$ 時間查詢組合數。

```
#include <vector>
```

```cpp
using namespace std;

const int MOD = 1e9 + 7;
const int MAXN = 200000;

long long fact[MAXN + 1], invFact[MAXN + 1];

void precompute() {
    fact[0] = 1;
    invFact[0] = 1;
    for (int i = 1; i <= MAXN; ++i) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }
    invFact[MAXN] = modInverse(fact[MAXN], MOD); // 呼叫 6.1 的 modInverse
    for (int i = MAXN - 1; i >= 1; --i) {
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
    }
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
```

---

7. 字串處理演算法 (String Algorithms)

7.1 KMP 字串匹配演算法 (Knuth-Morris-Pratt)

- 💡 **這在解決什麼問題？**
  在一大串文字 T 中，尋找有沒有出現子字串 P。
- 🧠 **直覺與核心思路：**
- **傳統暴力法的缺點**：如果比對到第 5 個字不符合，傳統做法會把主串指標往後退一位，重新從第一個字比對（產生大量無用重複比對）。
- **KMP 不走回頭路：**
- 當比對失敗時，我們其實**已經知道前面比對成功的幾個字長怎樣了**！
- **預處理 Failure Function (`pi` 陣列)**：算出 Pattern 字串中每個位置前綴與後綴的最長重疊長度。
- **智慧滑動**：當比對失敗時，主串 T 的指標完全不後退，只把 Pattern P 往右滑動到最長相同前綴的位置繼續比對。時間複雜度直接降到完美的 $O(N + M)$。
- **觀念**：建構前綴函數 `pi` 陣列（最長相等前後綴），實現 $O(N + M)$ 無回溯的單模式匹配。

```cpp
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// 計算 Pattern 的失敗函數 (Prefix Function)
vector<int> compute_pi(const string& p) {
    int m = p.length();
    vector<int> pi(m, 0);
    for (int i = 1, j = 0; i < m; ++i) {
        while (j > 0 && p[i] != p[j]) j = pi[j - 1];
        if (p[i] == p[j]) j++;
        pi[i] = j;
    }
    return pi;
}

// 在 text 中搜尋 pattern 出現的所有起始位置
vector<int> kmp_search(const string& text, const string& pattern) {
    vector<int> matches;
    vector<int> pi = compute_pi(pattern);
    int n = text.length(), m = pattern.length();

    for (int i = 0, j = 0; i < n; ++i) {
        while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
        if (text[i] == pattern[j]) j++;
        if (j == m) {
            matches.push_back(i - m + 1);
            j = pi[j - 1];
        }
    }
    return matches;
}
```

7.2 雙滾動雜湊 (Double String Hashing)

- 💡 **這在解決什麼問題？**
  想在 $O(1)$ 時間內，判斷兩個很長的字串（或任意子字串）是否完全一模一樣。
- 🧠 **直覺與核心思路**：
- **把字串變成數字指紋**：
- 把字串看成是一個「P 進位的超大數字」（例如字串 "abc" 換算成 $a \cdot P^2 + b \cdot P^1 + c \cdot P^0$），再對大質數 M 取模。
- 預處理前綴 Hash 之後，任何子字串 $S[l \dots r]$ 的 Hash 值，都可以像前綴和一樣減一減，在 $O(1)$ 時間算出！

- **什麼是雙 Hash？**：
- 萬一不同的字串算出來的 Hash 值剛好一樣（碰撞 Collision），就會判斷錯誤。
- **解決方案**：同時用**兩套不同的進位制與質數**（例如 (P_1, M_1) 與 (P_2, M_2)）各算一次。兩個指紋同時碰撞的機率低於中頭獎，可以視為 100% 準確！
- **觀念**：將字串轉為整數雜湊值，預處理後可在 $O(1)$ 時間比對任意子字串是否相等。

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

struct StringHash {
    int n;
    const long long P1 = 31, M1 = 1e9 + 7;
    const long long P2 = 37, M2 = 1e9 + 9;
    vector<long long> h1, h2, p1, p2;

    StringHash(const string& s) {
        n = s.length();
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);

        for (int i = 0; i < n; ++i) {
            h1[i + 1] = (h1[i] * P1 + (s[i] - 'a' + 1)) % M1;
            h2[i + 1] = (h2[i] * P2 + (s[i] - 'a' + 1)) % M2;
            p1[i + 1] = (p1[i] * P1) % M1;
            p2[i + 1] = (p2[i] * P2) % M2;
        }
    }

    // 取得子字串 s[l...r] (0-indexed) 的雙 Hash 值
    pair<long long, long long> get_hash(int l, int r) {
        long long hash1 = (h1[r + 1] - h1[l] * p1[r - l + 1]) % M1;
        if (hash1 < 0) hash1 += M1;

        long long hash2 = (h2[r + 1] - h2[l] * p2[r - l + 1]) % M2;
        if (hash2 < 0) hash2 += M2;

        return {hash1, hash2};
    }
};
```

這是一份針對 NotebookLM 整理之考點清單進行比對後，為你**補齊先前報告未詳細列出或實作之核心演算法**的進階實作補充集 。

這份內容專為 **YTP 決賽與高階程式競賽** 設計，包含了 **離線莫隊、雙堆疊、Bitset 圖論優化、擺動 DP、線性時間貪婪翻轉** 等重點演算法的詳細邏輯觀念與可直接套用的完整 C++ 範例程式碼。

---

8. 雙堆疊（Dual Heaps）動態維護第 K 大/小值與中位數

-

對應考古題：2020-Final Q7: kthrevenge

- **核心邏輯觀念**：
- 當資料流動態輸入，且需要頻繁查詢「當前第 K 大值」或「中位數」時，直接排序為 $O(N \log N)$ 會超時。
- **雙堆疊策略**：使用一個大頂堆（Max Heap）**維護較小的一半資料，一個**小頂堆（Min Heap）維護較大的一半資料。
- 保持大頂堆的大小恰好為 K。當新增元素時，依數值大小放入對應堆疊，並透過雙向彈出（Pop & Push）調整兩堆數量平衡。
- **複雜度**：每次插入 $O(\log N)$，每次查詢 $O(1)$。

```
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

// 動態維護前 K 小/大值的雙堆疊類別
class DualHeapKth {
private:
    priority_queue<int> max_heap; // 維護較小的部分，頂端為這部分的最大值
    priority_queue<int, vector<int>, greater<int>> min_heap; // 維護較大的部分，頂端為這部分的最小值
    int k;

public:
    DualHeapKth(int k_val) : k(k_val) {}

    // 新增數字並自動調整平衡
    void add(int val) {
        if (max_heap.empty() || val <= max_heap.top()) {
            max_heap.push(val);
        } else {
            min_heap.push(val);
        }

        // 保持 max_heap 的元素數量恰好等於 k
```

```
        while ((int)max_heap.size() > k) {
            min_heap.push(max_heap.top());
            max_heap.pop();
        }
        while ((int)max_heap.size() < k && !min_heap.empty()) {
            max_heap.push(min_heap.top());
            min_heap.pop();
        }
    }

    // 取得當前第 K 小的數值（若不足 K 個則傳回 -1）
    int getKth() {
        if ((int)max_heap.size() < k) return -1;
        return max_heap.top();
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N = 6, K = 3;
    vector<int> stream = {7, 2, 9, 4, 1, 6};
    DualHeapKth tracker(K);

    for (int x : stream) {
        tracker.add(x);
        cout << "加入 " << x << " 後的第 " << K << " 小值為: " << tracker.getKth()
<< "\n";
    }
    return 0;
}
```

---

9. 莫隊演算法（Mo's Algorithm）與樹狀陣列（Fenwick Tree / BIT）

-

對應考古題：2022-APAC: Cat Breed Statistics

- **核心邏輯觀念**：
- **莫隊演算法（Mo's Algorithm）**：處理無修改的**區間離線查詢**問題。將序列分成大小為 $B = \sqrt{N}$ 的區塊，把所有查詢 (L, R) 按照 (L / B, R) 排序。利用滑動雙指標 $[curL, curR]$ 以 $O(1)$ 或 $O(\log N)$ 轉移區間，將總時間複雜度優化至 $O((N+Q)\sqrt{N})$。
- **樹狀陣列（Fenwick Tree）**：高效率的單點修改與區間字首和查詢資料結構，常用於離線統計或配合莫隊演算法維護頻率。

- **複雜度**：莫隊 $O(N\sqrt{Q})$，樹狀陣列修改/查詢 $O(\log N)$。

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>

using namespace std;

int BLOCK_SIZE;

// 莫隊查詢結構體
struct Query {
    int l, r, id;
    bool operator<(const Query& other) const {
        int b1 = l / BLOCK_SIZE;
        int b2 = other.l / BLOCK_SIZE;
        if (b1 != b2) return b1 < b2;
        // 奇偶區塊擺動排序優化，減少 R 指標回跳距離
        return (b1 % 2 == 1) ? (r < other.r) : (r > other.r);
    }
};

// 樹狀陣列 (Fenwick Tree) 實作
struct FenwickTree {
    int n;
    vector<int> tree;
    FenwickTree(int n) : n(n), tree(n + 1, 0) {}

    void update(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }

    int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
};

// 莫隊演算法離線處理主函式
void solveMo(int N, int Q, const vector<int>& A, vector<Query>& queries) {
    BLOCK_SIZE = max(1, (int)(N / sqrt(Q)));
    sort(queries.begin(), queries.end());

    vector<int> freq(100005, 0);
    vector<int> ans(Q);
    int current_unique_count = 0; // 統計區間內相異元素的數量

    auto add = [&](int idx) {
        int val = A[idx];
        if (freq[val] == 0) current_unique_count++;
        freq[val]++;
```

```
    };

    auto remove = [&](int idx) {
        int val = A[idx];
        freq[val]--;
        if (freq[val] == 0) current_unique_count--;
    };

    int curL = 0, curR = -1;
    for (const auto& q : queries) {
        while (curL > q.l) add(--curL);
        while (curR < q.r) add(++curR);
        while (curL < q.l) remove(curL++);
        while (curR > q.r) remove(curR--);
        ans[q.id] = current_unique_count;
    }

    for (int i = 0; i < Q; i++) {
        cout << ans[i] << "\n";
    }
}
```

---

10. Bitset 優化圖論可達性與傳遞閉包（Transitive Closure）

-

對應考古題：2020-Final Q5: dependencies

- **核心邏輯觀念**：
- 在有向無環圖（DAG）中求「點 u 是否能到達點 v」（傳遞閉包/依賴關係判定），若使用傳統 DFS/BFS，總複雜度為 $O(V \cdot (V+E))$。
- **Bitset 加速**：利用 C++ 的 `std::bitset` 儲存每個點的可達點集合（第 k 位元為 1 表示可達點 k）。
- 按照**逆拓撲順序**推進，若存在邊 $u \to v$，則點 u 的可達集合等於 `reach[u] |= reach[v]`。這能將位元邏輯運算的常數壓縮 64 倍。
- **複雜度**：時間 $O\left(\frac{V \cdot E}{64}\right)$，空間 $O\left(\frac{V^2}{64}\right)$。

```
#include <iostream>
#include <vector>
#include <bitset>
#include <queue>

using namespace std;
```

```cpp
const int MAXN = 2005;
bitset<MAXN> reach[MAXN]; // reach[u][v] = 1 表示 u 可到達 v
vector<int> adj[MAXN];
int in_degree[MAXN];

void computeTransitiveClosure(int N) {
    queue<int> q;
    vector<int> topo_order;

    for (int i = 1; i <= N; i++) {
        reach[i].set(i); // 自己必可到達自己
        if (in_degree[i] == 0) q.push(i);
    }

    // 取得拓撲排序
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo_order.push_back(u);
        for (int v : adj[u]) {
            if (--in_degree[v] == 0) q.push(v);
        }
    }

    // 依逆拓撲序計算位元圖轉移
    for (int i = (int)topo_order.size() - 1; i >= 0; i--) {
        int u = topo_order[i];
        for (int v : adj[u]) {
            reach[u] |= reach[v]; // Bitset OR 操作，一次並集 64 個點
        }
    }
}

int main() {
    int N = 4;
    // 建立 1->2, 2->3, 3->4 依賴關係
    adj[1].push_back(2); in_degree[2]++;
    adj[2].push_back(3); in_degree[3]++;
    adj[3].push_back(4); in_degree[4]++;

    computeTransitiveClosure(N);

    cout << "1 是否能到達 4: " << (reach[1][4] ? "YES" : "NO") << "\n";
    cout << "3 是否能到達 1: " << (reach[3][1] ? "YES" : "NO") << "\n";
    return 0;
}
```

---

11. 閃電型 / 擺動子序列動態規劃（Wiggle / Zigzag

Subsequence DP)

-

對應考古題：2020-Final Q10: ramen

- **核心邏輯觀念**：
- 尋找最長的「升降交替」子序列（即：$a_1 < a_2 > a_3 < a_4 \ldots$）。
- **DP 狀態定義**：
- up：以當前元素結尾、且最後一步是**上升**的最長擺動子序列長度。
- down：以當前元素結尾、且最後一步是**下降**的最長擺動子序列長度。
- **狀態轉移方程**：
- 若 $A[i] > A[i-1]$：up = down + 1 （可將當前元素接在先前下降的末端）。
- 若 $A[i] < A[i-1]$：down = up + 1 （可將當前元素接在先前上升的末端）。
- 若 $A[i] == A[i-1]$：長度保持不變。
- **複雜度**：時間 $O(N)$，空間 $O(1)$。

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 計算最長擺動/閃電子序列長度
int maxWiggleLength(const vector<int>& nums) {
    int n = nums.size();
    if (n < 2) return n;

    int up = 1;   // 當前處於上升趨勢的最長長度
    int down = 1; // 當前處於下降趨勢的最長長度

    for (int i = 1; i < n; i++) {
        if (nums[i] > nums[i - 1]) {
            up = down + 1;
        } else if (nums[i] < nums[i - 1]) {
            down = up + 1;
        }
    }

    return max(up, down);
}

int main() {
```

```
    vector<int> ramen_ratings = {1, 7, 4, 9, 2, 5};
    cout << "最長閃電型子序列長度: " << maxWiggleLength(ramen_ratings) << "\n"; //
輸出 6
    return 0;
}
```

---

12. 線性時間滑動視窗/差分強制翻轉策略（Greedy K-Bit Flip）

-

對應考古題：2020-Final Q6: pancake

- **核心邏輯觀念**：
- 給定一個由 0 和 1 組成的陣列，每次可以選擇長度為 K 的子區間進行翻轉（0變1，1變0），求將全部元素變成 1 的最少操作次數。
- **貪婪原則**：由左至右遍歷，當碰到第一個 0 時，因為左側已無其他操作能影響它，**必須**在此位置發起一次長度為 K 的翻轉。
- **差分/佇列優化**：如果每次實作真正去翻轉 K 個元素，複雜度會暴增至 $O(N \cdot K)$。我們維護一個長度為 K 的滑動視窗翻轉狀態標記，以 $O(1)$ 時間轉移，將整體優化至 $O(N)$。
- **複雜度**：時間 $O(N)$，空間 $O(N)$。

```
#include <iostream>
#include <vector>

using namespace std;

// 求將所有位元翻轉為 1 的最少長度 K 視窗翻轉次數
int minKBitFlips(vector<int>& A, int K) {
    int N = A.size();
    vector<int> isFlipped(N, 0); // 記錄 i 位置是否為發起翻轉的起點
    int current_flips = 0;       // 當前位置覆蓋到的翻轉次數
    int total_flips = 0;

    for (int i = 0; i < N; i++) {
        // 當視窗移出 K 範圍時，扣除 K 步前的翻轉影響
        if (i >= K) {
            current_flips ^= isFlipped[i - K];
        }

        // 當前位元的實際數值 = 原始值 ^ 當前翻轉奇偶性
        if ((A[i] ^ (current_flips & 1)) == 0) {
            // 實際值為 0，必須在此處發起長度 K 的翻轉
```

```
            if (i + K > N) return -1; // 剩餘長度不足 K，無法達成目標

            isFlipped[i] = 1;
            current_flips ^= 1;
            total_flips++;
        }
    }

    return total_flips;
}

int main() {
    vector<int> pancakes = {0, 0, 0, 1, 0, 1, 1, 0};
    int K = 3;
    int result = minKBitFlips(pancakes, K);
    cout << "最少翻轉次數: " << result << "\n";
    return 0;
}
```

---

13. 網格模擬通用範例與高速 I/O 輸入框架

-

對應考古題：2020-Final Q0: solution, Q1: minesweeper

- **核心邏輯觀念**：
- **標準輸入優化**：決賽可能面臨數十萬筆輸入輸出，關閉 cin/cout 同步可以顯著縮短運行時間。
- **網格方向陣列 (Offset Arrays)**：處理 4 方向或 8 方向地圖遍歷時，嚴禁撰寫大量重複的 if-else，應統一使用 dr[] 與 dc[] 進行迴圈掃描與邊界檢查。
- **複雜度**：$O(R \cdot C)$。

```
#include <iostream>
#include <vector>

using namespace std;

// 8 方向偏移量 (上下左右、四個對角線)
const int dr[8] = {-1, -1, -1,  0, 0,  1, 1, 1};
const int dc[8] = {-1,  0,  1, -1, 1, -1, 0, 1};

// 邊界檢查函式
inline bool isValid(int r, int c, int R, int C) {
    return (r >= 0 && r < R && c >= 0 && c < C);
}
```

```cpp
void solveMinesweeper() {
    int R, C;
    if (!(cin >> R >> C)) return;

    vector<vector<int>> grid(R, vector<int>(C));
    vector<vector<int>> ans(R, vector<int>(C, 0));

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            cin >> grid[i][j];
        }
    }

    // 計算 8 相鄰地雷數
    for (int r = 0; r < R; r++) {
        for (int c = 0; c < C; c++) {
            for (int d = 0; d < 8; d++) {
                int nr = r + dr[d];
                int nc = c + dc[d];
                if (isValid(nr, nc, R, C) && grid[nr][nc] == 1) {
                    ans[r][c]++;
                }
            }
        }
    }

    // 輸出結果
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            cout << ans[i][j] << (j == C - 1 ? "" : " ");
        }
        cout << "\n";
    }
}

int main() {
    // 競賽加速模板
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solveMinesweeper();
    return 0;
}
```

💡 競賽現場解題速查表 (Checklist & Reminders)

1. **輸入輸出加速**：第一行為必寫範式：

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

2. **防溢位 (Overflow)**：涉及乘法或路徑長度加總時，一律使用 `long long`；無限大常量請開 `1e18` 而非 `0x3f3f3f3f`。
3. **二分搜邊界**：確認為 `low <= high` 或 `low < high`，防範 `mid` 陷入死循環（常用 `mid = low + (high - low) / 2`）。
4. **樹與圖陣列**：大小請習慣開到 `N + 5` 預留緩衝，空間限制充足時線段樹需開 `4 * N`。
5. **先判斷題型**：

- 看資料範圍 N：$N \leq 20 \implies$ 狀壓 DP 或窮舉；$N \leq 10^5 \implies O(N \log N)$ 的二分搜、線段樹、Dijkstra；$N \leq 5000 \implies O(N^2)$ DP。

6. **拿分優先順序**：

- 寫出正確的暴力搜 (DFS/BFS) 拿到子任務基本分。
- 檢查有沒有單調性，有的話立馬套用 **二分搜答案** 樣板。
- 圖論題先建鄰接表，路徑題直接套 **Dijkstra**。

7. **資料結構選用指南**：

- 頻率/區間查詢 + 無修改 $\implies$ **莫隊演算法**。
- 動態加入 + 求第 K 大 / 中位數 $\implies$ **雙堆疊（Dual Heap）**。
- 單點修改 + 區間和 $\implies$ **樹狀陣列（Fenwick Tree）**。

8. **圖論與 DP 優化常數技巧**：

- 圖論點數 $N \leq 2000$ 且問連通性/可達性 $\implies$ 用 `std::bitset` 將可達性陣列進行常數壓縮。
- 連續升降狀態轉移 DP $\implies$ 分拆 `up` 與 `down` 兩個狀態標記。